

CSDS 2026 – Week 10

Remark: Using AI to improve the plot formatting and interpretation.

Initialization

```
set.seed(20260429)
```

Let's go back and take another look at our analysis of the cars dataset. Recall our variables:

No.	Variable	Description
1	mpg	miles-per-gallon (dependent variable)
2	cylinders	cylinders in engine
3	displacement	size of engine
4	horsepower	power of engine
5	weight	weight of car
6	acceleration	acceleration ability of car
7	model_year	year model was released
8	origin	place car was designed (1: USA, 2: Europe, 3: Japan)

```
auto <- read.table("auto-data.txt", header = FALSE, na.strings = "?")
auto <- na.omit(auto)
names(auto) <- c("mpg", "cylinders", "displacement", "horsepower", "weight",
                 "acceleration", "model_year", "origin", "car_name")
cars <- auto
```

Question 1

Let's deal with **nonlinearity** first. Create a new dataset that log-transforms several variables from our original dataset (called `cars` in this case):

```
cars_log <- with(cars,
                 data.frame(log(mpg), log(cylinders), log(displacement),
                             log(horsepower), log(weight), log(acceleration),
                             model_year, origin))
```

```
names(cars_log)
```

```
## [1] "log.mpg."      "log.cylinders." "log.displacement."
## [4] "log.horsepower." "log.weight."    "log.acceleration."
## [7] "model_year"    "origin"
```

original model:

```
model_mpg <- lm(mpg ~ cylinders + displacement + horsepower + weight +
                acceleration + model_year + factor(origin),
                data = cars)
summary(model_mpg)
```

```
##
```

```
## Call:
```

```
## lm(formula = mpg ~ cylinders + displacement + horsepower + weight +
```

```
##      acceleration + model_year + factor(origin), data = cars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -9.0095 -2.0785 -0.0982  1.9856 13.3608
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.795e+01  4.677e+00  -3.839 0.000145 ***
## cylinders      -4.897e-01  3.212e-01  -1.524 0.128215
## displacement   2.398e-02  7.653e-03   3.133 0.001863 **
## horsepower     -1.818e-02  1.371e-02  -1.326 0.185488
## weight         -6.710e-03  6.551e-04 -10.243 < 2e-16 ***
## acceleration    7.910e-02  9.822e-02   0.805 0.421101
## model_year      7.770e-01  5.178e-02 15.005 < 2e-16 ***
## factor(origin)2 2.630e+00  5.664e-01   4.643 4.72e-06 ***
## factor(origin)3 2.853e+00  5.527e-01   5.162 3.93e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.307 on 383 degrees of freedom
## Multiple R-squared:  0.8242, Adjusted R-squared:  0.8205
## F-statistic: 224.5 on 8 and 383 DF, p-value: < 2.2e-16
```

a. Run a new regression on the cars_log dataset, with mpg.log. dependent on all other variables

- i. Which log-transformed factors have a significant effect on log.mpg. at 10% significance?

```
regr_log <- lm(log.mpg. ~ log.cylinders. + log.displacement. + log.horsepower. +
               log.weight. + log.acceleration. + model_year +
               factor(origin), data = cars_log)
summary(regr_log)
```

```
##
## Call:
## lm(formula = log.mpg. ~ log.cylinders. + log.displacement. +
##     log.horsepower. + log.weight. + log.acceleration. + model_year +
##     factor(origin), data = cars_log)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.39727 -0.06880  0.00450  0.06356  0.38542
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7.301938   0.361777  20.184 < 2e-16 ***
## log.cylinders. -0.081915   0.061116  -1.340 0.18094
## log.displacement. 0.020387   0.058369   0.349 0.72707
## log.horsepower. -0.284751   0.057945  -4.914 1.32e-06 ***
## log.weight.     -0.592955   0.085165  -6.962 1.46e-11 ***
## log.acceleration. -0.169673   0.059649  -2.845 0.00469 **
## model_year      0.030239   0.001771 17.078 < 2e-16 ***
## factor(origin)2  0.050717   0.020920   2.424 0.01580 *
## factor(origin)3  0.047215   0.020622   2.290 0.02259 *
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.113 on 383 degrees of freedom
## Multiple R-squared:  0.8919,    Adjusted R-squared:  0.8897
## F-statistic:   395 on 8 and 383 DF,  p-value: < 2.2e-16
```

- ii. Do some new factors now have effects on mpg, and why might this be?

Yes. `log(horsepower)` and `log(acceleration)` are significant in the log model, while they were not significant in the original model.

This is likely because the log transformation changes the functional form, improves linearity, and reduces the effect of heteroskedasticity or outliers.

- Which factors still have insignificant or opposite (from correlation) effects on mpg? Why might this be?

- `log(cylinders)` remains insignificant.
- `log(displacement)` becomes insignificant and has a sign opposite to its simple correlation with mpg.

This is likely caused by multicollinearity among the explanatory variables. Since variables such as cylinders, displacement, horsepower, and weight are highly correlated, the estimated partial effect of one variable can become insignificant or even change sign once the others are controlled for.

b. Let's take a closer look at weight, because it seems to be a major explanation of mpg

- i. Create a regression (call it `regr_wt`) of mpg over weight from the original cars dataset

```
regr_wt <- lm(mpg ~ weight, data = cars)
summary(regr_wt)

##
## Call:
## lm(formula = mpg ~ weight, data = cars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.9736  -2.7556  -0.3358   2.1379  16.5194
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  46.216524   0.798673   57.87  <2e-16 ***
## weight      -0.007647   0.000258  -29.64  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.333 on 390 degrees of freedom
## Multiple R-squared:  0.6926,    Adjusted R-squared:  0.6918
## F-statistic: 878.8 on 1 and 390 DF,  p-value: < 2.2e-16
```

- ii. Create a regression (call it `regr_wt_log`) of `log.mpg.` on `log.weight.` from `cars_log`

```
regr_wt_log <- lm(log.mpg. ~ log.weight., data = cars_log)
summary(regr_wt_log)

##
## Call:
## lm(formula = log.mpg. ~ log.weight., data = cars_log)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.52321 -0.10446 -0.00772  0.10124  0.59445
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  11.5152     0.2365   48.69  <2e-16 ***
## log.weight.  -1.0575     0.0297  -35.61  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1651 on 390 degrees of freedom
## Multiple R-squared:  0.7648,    Adjusted R-squared:  0.7642
## F-statistic: 1268 on 1 and 390 DF,  p-value: < 2.2e-16
```

- iii. Visualize the residuals (density plot & scatterplot of log.weight. vs. residuals) of both regression models (raw and log-transformed):

```
plot_hist_density <- function(res, main_title,
                              hist_col = "#ecf5f6",
                              den_col = "#4646f2",
                              ref_col = "#e24545") {

  d <- density(res)
  h <- hist(res, plot = FALSE, probability = TRUE)
  ymax <- max(h$density, d$y) * 1.15

  hist(res,
        probability = TRUE,
        ylim = c(0, ymax),
        main = main_title,
        xlab = "Residuals",
        col = hist_col,
        border = "white")
  lines(d, col = den_col, lwd = 2)
  abline(v = 0, col = ref_col, lwd = 2)
}

plot_res_scatter <- function(x, res, main_title, xlab_title,
                             ref_col = "#e24545") {

  plot(x, res,
        main = main_title,
        xlab = xlab_title,
        ylab = "Residuals",
        pch = 19, cex = 0.6)
  abline(h = 0, col = ref_col, lwd = 2)
}

par(mfrow = c(2, 2), cex.main = 0.95, cex.lab = 0.9, cex.axis = 0.85)

res_raw <- residuals(regr_wt)
res_log <- residuals(regr_wt_log)

plot_hist_density(res_raw, "Raw Residual Distribution")

## Warning in hist.default(res, plot = FALSE, probability = TRUE): argument
```

```
## 'probability' is not made use of
```

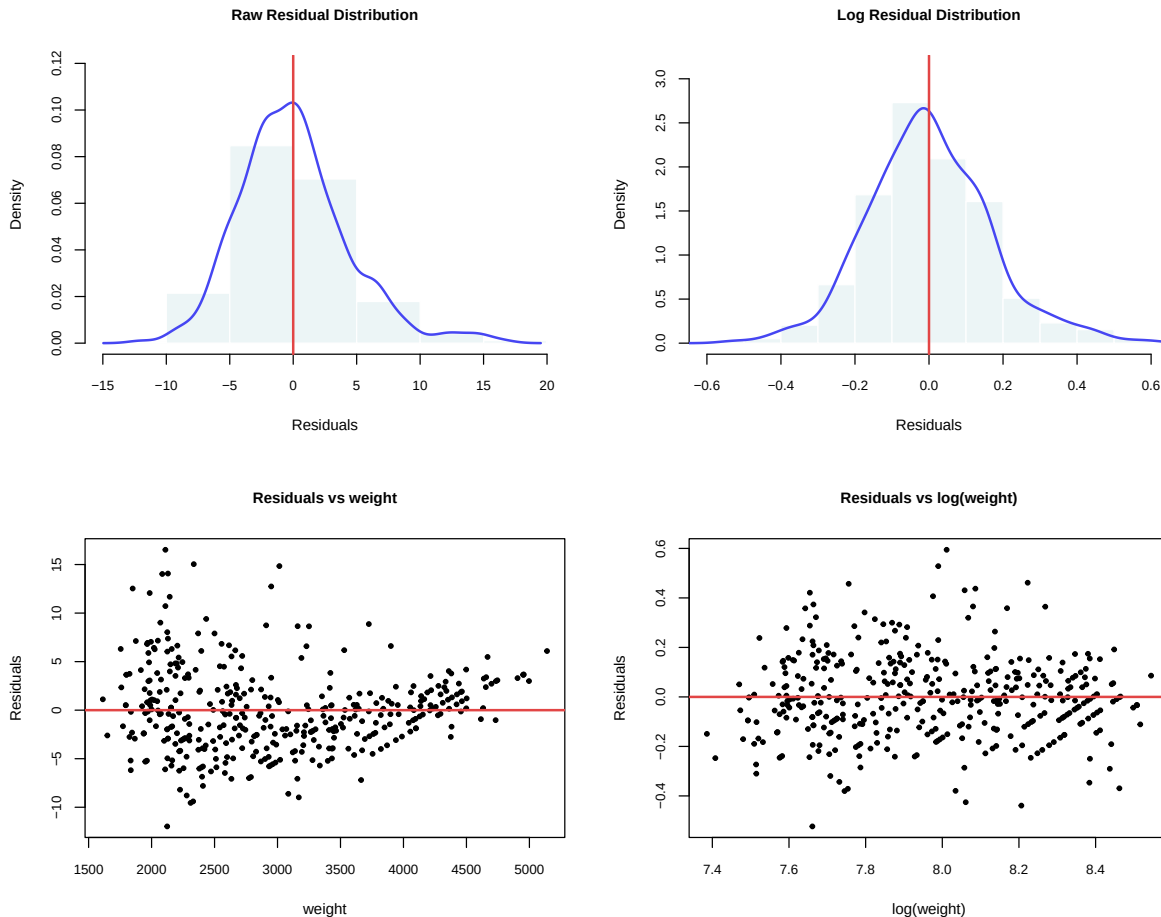
```
plot_hist_density(res_log, "Log Residual Distribution")
```

```
## Warning in hist.default(res, plot = FALSE, probability = TRUE): argument
```

```
## 'probability' is not made use of
```

```
plot_res_scatter(cars$weight, res_raw,
                 "Residuals vs weight", "weight")
```

```
plot_res_scatter(cars_log$log.weight., res_log,
                 "Residuals vs log(weight)", "log(weight)")
```



- iv. Which regression produces better distributed residuals for the assumptions of regression?

The log-transformed regression `regr_wt_log` produces better distributed residuals. The residuals from the log model are more symmetrically distributed around zero and show less skewness compared to the raw model `regr_wt`.

- v. How would you interpret the slope of `log.weight.` vs `log.mpg.` in simple words?

The slope tells us the percent change in `mpg` for a 1% increase in `weight`. In this case, the coefficient of `log(weight)` is -1.0575, then a 1% increase in `weight` is associated with about a 1.0575% decrease in `mpg` on average, holding the other variables fixed.

- vi. From its standard error, what is the 95% confidence interval of the slope of `log.weight.` vs `log.mpg.`?

```

coef_est <- coef(regr_wt_log)["log.weight."]
se_est <- summary(regr_wt_log)$coefficients["log.weight.", "Std. Error"]
ci_lower <- coef_est - 1.96 * se_est
ci_upper <- coef_est + 1.96 * se_est
cat("95% CI: [", ci_lower, ", ", ci_upper, "]\n")
## 95% CI: [ -1.115715 , -0.9992977 ]

```

Question 2

Let's tackle **multicollinearity** next. Consider the regression model:

```

regr_log <- lm(log.mpg.
              ~ log.cylinders. + log.displacement. + log.horsepower. +
                log.weight. + log.acceleration. + model_year +
                factor(origin), data = cars_log)

```

a. Using regression and R2, compute the VIF of log.weight. using the approach shown in class

```

weight_regr <- lm(log.weight. ~ log.cylinders. + log.displacement. + log.horsepower. +
                  log.acceleration. + model_year + factor(origin), data = cars_log)

r2_weights <- summary(weight_regr)$r.squared
vif_weights <- 1 / (1 - r2_weights)
vif_weights
## [1] 17.57512

```

b. Use Stepwise VIF Selection to remove highly collinear predictors

- i. Use vif(regr_log) to compute VIF of the all the independent variables

```

library(car)
## Loading required package: carData

vif_scores <- vif(regr_log)
vif_scores
##
##          GVIF Df GVIF^(1/(2*Df))
## log.cylinders. 10.456738 1      3.233688
## log.displacement. 29.625732 1      5.442952
## log.horsepower. 12.132057 1      3.483110
## log.weight. 17.575117 1      4.192269
## log.acceleration. 3.570357 1      1.889539
## model_year 1.303738 1      1.141814
## factor(origin) 2.656795 2      1.276702

```

- ii. Eliminate from your model the single independent variable with the largest VIF score that is also greater than 5

For predictors with 1 degree of freedom, I used GVIF as the VIF measure. For predictors with more than 1 degree of freedom, I used $GVIF^{1/(2 \cdot Df)}$ for comparison. Then I removed the variable with the largest VIF greater than 5 at each step.

```

remove_max_vif <- function(model, threshold = 5) {
  vif_scores <- vif(model)

  vif_metric <- ifelse(
    vif_scores[, "Df"] == 1,
    vif_scores[, "GVIF"],
    vif_scores[, "GVIF^(1/(2*Df))"]
  )

  max_vif_var <- rownames(vif_scores)[which.max(vif_metric)]
  max_vif_val <- max(vif_metric)
  removed_var <- ifelse(max_vif_val > threshold, max_vif_var, NA)

  if (!is.na(removed_var)) {
    cat("Removed variable:", removed_var,
        "| VIF =", round(max_vif_val, 4), "\n")

    updated_model <- update(
      model,
      as.formula(paste(". ~ . -", removed_var))
    )
  } else {
    cat("No variable has VIF greater than", threshold, "\n")
    updated_model <- model
  }

  list(
    model = updated_model,
    vif_scores = vif_scores,
    vif_metric = vif_metric,
    max_vif_var = max_vif_var,
    max_vif_val = max_vif_val,
    removed_var = removed_var
  )
}

```

```
regr_log_updated <- remove_max_vif(regr_log)$model
```

```
## Removed variable: log.displacement. | VIF = 29.6257
```

```
regr_log_updated
```

```
##
```

```
## Call:
```

```
## lm(formula = log.mpg. ~ log.cylinders. + log.horsepower. + log.weight. +
##     log.acceleration. + model_year + factor(origin), data = cars_log)
```

```
##
```

```
## Coefficients:
```

```
##      (Intercept)      log.cylinders.      log.horsepower.      log.weight.
##           7.26400          -0.06712          -0.28552          -0.57510
## log.acceleration.      model_year      factor(origin)2      factor(origin)3
##          -0.17510           0.03018           0.04717           0.04394
```

- iii. Repeat steps (i) and (ii) until no more independent variables have VIF scores above 5

```

step <- 1

while (TRUE) {
  cat("\nStep", step, "\n")
  result <- remove_max_vif(regr_log_updated)

  if (is.na(result$removed_var)) {
    break
  }

  regr_log_updated <- result$model
  step <- step + 1
}

##
## Step 1
## Removed variable: log.horsepower. | VIF = 12.1145
##
## Step 2
## Removed variable: log.cylinders. | VIF = 5.4276
##
## Step 3
## No variable has VIF greater than 5 .

```

- iv. Report the final regression model and its summary statistics

```

summary(regr_log_updated)

##
## Call:
## lm(formula = log.mpg. ~ log.weight. + log.acceleration. + model_year +
##     factor(origin), data = cars_log)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.38259 -0.07054  0.00401  0.06696  0.39798
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7.410974   0.316806  23.393 < 2e-16 ***
## log.weight.   -0.875499   0.029086 -30.101 < 2e-16 ***
## log.acceleration. 0.054377   0.037132   1.464 0.14389
## model_year     0.032787   0.001731  18.937 < 2e-16 ***
## factor(origin)2  0.056111   0.018241   3.076 0.00225 **
## factor(origin)3  0.031937   0.018506   1.726 0.08519 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1163 on 386 degrees of freedom
## Multiple R-squared:  0.8845,    Adjusted R-squared:  0.883
## F-statistic: 591.1 on 5 and 386 DF,  p-value: < 2.2e-16

```

The final regression model after stepwise VIF selection is:

$$\log(\text{mpg}) = \beta_0 + \beta_1 \log(\text{weight}) + \beta_2 \log(\text{acceleration}) + \beta_3 \text{model_year} + \beta_4 I(\text{origin} = 2) + \beta_5 I(\text{origin} = 3) + \epsilon$$

The model has $R^2 = 0.8845$ and adjusted $R^2 = 0.8830$, with residual standard error 0.1163. The highly collinear variables `log.displacement.`, `log.horsepower.`, and `log.cylinders.` were removed by stepwise VIF selection. All remaining variables except `log.acceleration.` are significant at the 10% level.

c. Using stepwise VIF selection, have we lost any variables that were previously significant? If so, how much did we hurt our explanation by dropping those variables? (hint: look at model fit)

We lost `log.horsepower.`, which was statistically significant in the full log-transformed model. After stepwise VIF selection, the model fit became slightly worse, but only by a small amount.

Specifically, R^2 decreased from 0.8919 to 0.8845, and adjusted R^2 decreased from 0.8897 to 0.8830. The residual standard error increased slightly from 0.1130 to 0.1163. This suggests that dropping the collinear variables reduced explanatory power only a little, while helping to reduce multicollinearity.

d. From only the formula for VIF, try deducing/deriving the following:

- i. If an independent variable has no correlation with other independent variables, what would its VIF score be?**

Let j denote the j -th independent variable. If this variable has no correlation with the other independent variables, then the R_j^2 value from regressing that variable on the others would be zero. Therefore, the VIF score would be:

$$\text{VIF}_j = \frac{1}{1 - R_j^2} = \frac{1}{1 - 0} = 1 \quad (1)$$

So the VIF score would be 1, indicating no multicollinearity.

- ii. Given a regression with only two independent variables (X1 and X2), how correlated would X1 and X2 have to be, to get VIF scores of 5 or higher? To get VIF scores of 10 or higher?**

Let j denote either X_1 or X_2 . For a regression with only two independent variables, the VIF for each variable can be calculated as:

$$\text{VIF}_j = \frac{1}{1 - R_j^2} \quad (2)$$

where R_j^2 is the squared correlation between X_1 and X_2 .

To get a VIF score of 5 or higher:

$$5 = \frac{1}{1 - R_j^2} \Rightarrow 1 - R_j^2 = \frac{1}{5} \Rightarrow R_j^2 = 1 - \frac{1}{5} = \frac{4}{5} = 0.8 \quad (3)$$

Therefore, X1 and X2 would need to have a correlation of at least $\sqrt{0.8} \approx 0.8944$ to get VIF scores of 5 or higher.

To get a VIF score of 10 or higher:

$$10 = \frac{1}{1 - R_j^2} \Rightarrow 1 - R_j^2 = \frac{1}{10} \Rightarrow R_j^2 = 1 - \frac{1}{10} = \frac{9}{10} = 0.9 \quad (4)$$

Therefore, X1 and X2 would need to have a correlation of at least $\sqrt{0.9} \approx 0.9487$ to get VIF scores of 10 or higher.

Question 3

Might the relationship of weight on mpg be different for cars from different origins?

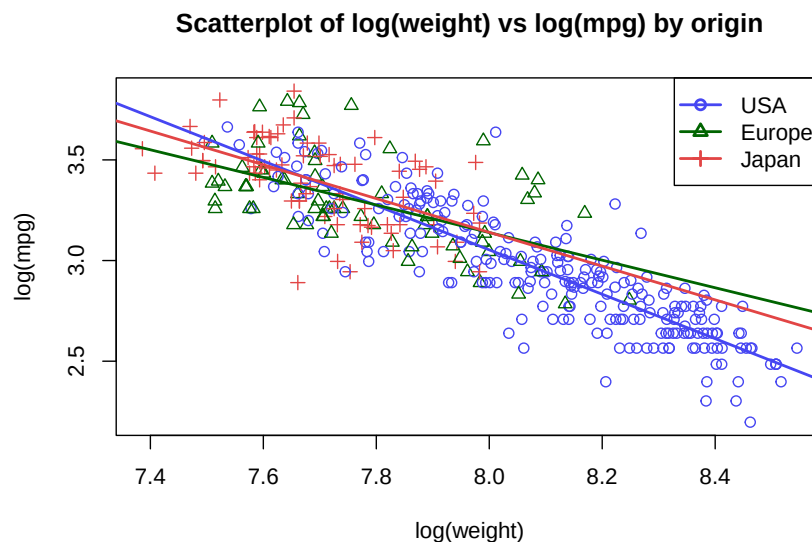
Let's try visualizing this. First, plot all the weights, using different colors and symbols for the three origins:

a. Let's add three separate regression lines on the scatterplot, one for each of the origins.

```
origin_colors <- c("#4646f2", "darkgreen", "#e24545")
with(cars_log,
     plot(log.weight., log.mpg., pch = origin, col = origin_colors[origin],
          xlab = "log(weight)", ylab = "log(mpg)",
          main = "Scatterplot of log(weight) vs log(mpg) by origin"))

for (i in 1:3) {
  abline(lm(log.mpg. ~ log.weight., data = subset(cars_log, origin == i)),
        col = origin_colors[i], lwd = 2)
}

legend("topright", legend = c("USA", "Europe", "Japan"),
      col = origin_colors, pch = 1:3, lwd = 2)
```



b. [Not Graded] Do cars from different origins appear to have different weight vs. mpg relationships?

The relationships look broadly similar negative slopes across origins, since all three regression lines slope downward. However, there are some differences in the positions of the lines, suggesting that cars from different origins may have somewhat different mpg levels at a given weight.